LAPORAN TUGAS PENGUJIAN PERANGKAT LUNAK
EKSPLORASI & IMPLEMENTASI SELENIUM AUTOMATION

Topik Bahasan	Selenium Web Automation & Software Testing Framework (Selenium IDE)
Nama / NIM	- Muhammad Rayhan Ramadhan (12409011010003) - Arya Putra Fahreza (12409011010005) - Azka Azfar (12409011010008)

BAB I: ANALISIS KONSEPTUAL SELENIUM (5W + 1H)

Selenium merupakan salah satu ekosistem pengujian otomatis (automated software testing) paling populer di dunia rekayasa perangkat lunak modern. Untuk memahami arsitektur, sejarah, dan cara kerjanya secara komprehensif, berikut adalah penjabaran mendalam menggunakan pendekatan analisis 5W + 1H:

Dimensi 5W+1H	Penjelasan Lengkap & Arsitektur Teknis
1. WHAT (Apa itu Selenium?)	Selenium adalah kerangka kerja (framework) open-source portabel yang digunakan untuk mengotomatisasi interaksi pada peramban web (web browser). Selenium bukanlah satu alat tunggal, melainkan sebuah suite perangkat lunak yang terdiri dari empat komponen utama: • Selenium IDE (Integrated Development Environment): Aplikasi/ekstensi berbasis Record & Playback untuk merekam interaksi pengguna di browser dan menjalankan pengujian otomatis tanpa perlu menulis kode dari nol. • Selenium RC (Remote Control / Selenium 1): Pendekatan generasi awal yang menyuntikkan JavaScript (Selenium Core) ke dalam browser melalui proxy server (kini telah digantikan oleh WebDriver). • Selenium WebDriver (Selenium 2/3/4): Komponen inti modern yang berkomunikasi langsung dengan browser secara native melalui protokol standar W3C WebDriver. • Selenium Grid: Komponen infrastruktur yang memungkinkan eksekusi pengujian secara paralel dan terdistribusi di berbagai mesin (node), sistem operasi, dan versi browser secara bersamaan. Selain itu, ekosistem Selenium mendukung berbagai bahasa pemrograman populer seperti Python, Java, C#, JavaScript (Node.js), Ruby, dan Kotlin.
2. WHY (Mengapa Menggunakan Selenium?)	Pengujian perangkat lunak secara manual (manual testing) pada aplikasi web berskala besar memakan waktu lama, rentan terhadap kesalahan manusia (human error), dan tidak efisien ketika dilakukan berulang kali pada setiap rilis

	fitur baru (regression testing). Selenium dipilih karena alasan berikut: • Open-Source & Gratis: Tidak memerlukan biaya lisensi komersial yang mahal seperti TestComplete atau UFT. • Mudah Dipelajari dengan Selenium IDE: Memungkinkan tester merekam alur pengujian secara visual (Record & Playback) dan mengekspornya ke berbagai bahasa pemrograman bila diperlukan. • Cross-Browser & Cross-Platform: Pengujian dapat dijalankan di Google Chrome, Mozilla Firefox, Microsoft Edge, dan Apple Safari di atas Windows, Linux, maupun macOS. • Efisiensi Waktu & Akurasi: Mampu memangkas waktu pengujian regresi yang berulang menjadi hitungan detik dengan konsistensi eksekusi yang tinggi.
3. WHO (Siapa Pencipta dan Pengguna Selenium?)	• Pencipta & Kontributor: Selenium pertama kali diciptakan oleh Jason Huggins pada tahun 2004 ketika bekerja sebagai engineer di ThoughtWorks. Selanjutnya, Shinya Kasatani menciptakan Selenium IDE pada tahun 2006 untuk mempercepat pembuatan test case melalui fitur Record & Playback, sementara Simon Stewart mengembangkan WebDriver. Saat ini Selenium dikelola oleh Software Freedom Conservancy bersama komunitas global. • Pengguna: Digunakan secara luas oleh Quality Assurance (QA) Engineer, Software Tester, Mahasiswa & Peneliti Rekayasa Perangkat Lunak, hingga Software Developer untuk memvalidasi fitur aplikasi web sebelum dirilis ke publik.
4. WHEN (Kapan Selenium Diciptakan dan Kapan Sebaiknya Digunakan?)	• Sejarah Perkembangan: Dimulai pada tahun 2004 (Selenium Core), disusul peluncuran Selenium IDE (2006), penggabungan WebDriver menjadi Selenium 2.0 (2011), adopsi standar W3C pada Selenium 3.0 (2016), hingga rilis Selenium 4.0 (2021–sekarang) yang menghadirkan aplikasi desktop Selenium IDE modern berbasis Electron. • Waktu Penggunaan dalam Siklus SDLC: Sangat ideal digunakan pada tahap Regression Testing (ketika fitur inti sudah stabil dan sering diuji ulang), Smoke & Sanity Testing sebelum deployment, serta End-to-End (E2E) Functional Testing pada alur bisnis utama seperti Login, Keranjang Belanja, dan Checkout.
5. WHERE (Di Mana Selenium Dapat Dijalankan?)	Selenium dapat dijalankan di berbagai lingkungan eksekusi: • Lingkungan Lokal (Local Machine): Langsung di laptop/PC pengembang menggunakan aplikasi desktop Selenium IDE Client atau ekstensi browser. • Command-Line Runner & Server: Proyek Selenium IDE (file .side) dapat dijalankan melalui terminal atau server CI/CD menggunakan alat resmi bernama selenium-side-runner. • Cloud Testing Platforms: Terintegrasi dengan layanan cloud pengujian seperti BrowserStack, Sauce Labs, dan LambdaTest.
6. HOW (Bagaimana Cara Kerja Selenium IDE?)	Mekanisme kerja Selenium IDE mengikuti tiga tahapan utama berbasis Record & Playback: 1. Perekaman Aksi (Recording): Ketika tombol Record ditekan, Selenium IDE membuka jendela browser dan mendengarkan setiap aksi pengguna pada

	Document Object Model (DOM) halaman web (seperti membuka halaman, mengklik elemen, atau mengetik teks). 2. Pembentukan Tabel Selenese (Command, Target, Value): Setiap interaksi secara otomatis diterjemahkan menjadi baris instruksi yang terdiri dari tiga komponen: Command (jenis perintah seperti open, click, type), Target (locator elemen seperti id=user-name atau css=.login-button), dan Value (nilai input yang diketikkan). 3. Pemutaran Ulang & Validasi (Playback & Assertion): Ketika tombol Play ditekan, Selenium IDE mengeksekusi ulang seluruh daftar perintah tersebut ke browser secara otomatis dan menandai setiap langkah dengan indikator warna Hijau (Passed/Berhasil) atau Merah (Failed/Gagal).

BAB II: KAJIAN SKRIPSI & TESIS LUAR NEGERI TENTANG SELENIUM

Berikut adalah penelitian skripsi (Bachelor's Thesis) dan tesis (Master's Thesis) dari berbagai perguruan tinggi di luar negeri (Swedia, Finlandia, dan Estonia) yang meneliti penerapan, evaluasi performa, serta pengembangan kerangka kerja pengujian otomatis menggunakan ekosistem Selenium:

No	Judul Skripsi/Tesis & Penulis	Universitas & Negara	Fokus Penelitian & Temuan Utama
1	Automated Functional Tests for a Web Application Penulis: Johan Wulf (2022)	Lund University, Swedia (Bachelor's Thesis in Computer Science)	Fokus: Otomatisasi pengujian fungsional antarmuka web (UI) pada aplikasi internal perusahaan ritel global IKEA (aplikasi MIA) dengan menginvestigasi dan membandingkan integrasi Selenium serta framework pengujian modern ke dalam alur pengembangan perangkat lunak. Hasil: Penelitian ini merancang alur pengujian otomatis (automated test flow) yang dieksekusi setiap kali pengembang melakukan push kode ke repositori. Studi ini mengevaluasi bagaimana Selenium memfasilitasi pemilihan elemen DOM melalui Locators & Finders serta kombinasinya dengan pustaka assertion untuk memvalidasi fitur pembuatan slot waktu secara otomatis. Tautan: https://lup.lub.lu.se/student-papers/record/9092785
2	Automated Web-Based Testing Framework for the Interface of a Power Plant Control Penulis: Duong Truong (2025)	Aalto University, Finlandia (Master's Thesis in Electrical and Automation Engineering)	Fokus: Perancangan kerangka kerja pengujian otomatis berbasis web untuk antarmuka Human-Machine Interface (HMI) pada sistem kontrol pembangkit listrik yang bersifat safety-critical. Hasil: Melakukan analisis komparatif antara Selenium dan Playwright dalam lingkungan

			industri berkecepatan tinggi serta integrasi CI/CD. Hasil tesis menunjukkan bahwa ekosistem matang Selenium unggul dalam dukungan lintas-browser yang luas, namun memerlukan strategi sinkronisasi yang tepat ketika menangani antarmuka HMI yang sangat dinamis. Tautan: https://aaltodoc.aalto.fi/bitstreams/05ae4267-c6b8-439d-859d-6f20cb0455c6/download
3	Choosing the Right Automated UI Testing Tool: A Comparative Study of Selenium and TestComplete Penulis: Sofia Tibell & Muskan Kholi (2023)	Linnaeus University (Linnéuniversitetet), Swedia (Bachelor's Thesis in Software Technology)	Fokus: Studi komparatif empiris antara alat pengujian open-source (Selenium) dan alat pengujian komersial berbayar (TestComplete) untuk pengujian antarmuka pengguna (UI). Hasil: Menguji kedua alat berdasarkan parameter efisiensi waktu pembuatan pengujian, kemudahan pemeliharaan (maintainability), kecepatan eksekusi, dan fleksibilitas integrasi. Hasil penelitian membuktikan bahwa Selenium memberikan fleksibilitas tinggi dan bebas biaya lisensi dibanding alat komersial. Tautan: https://www.diva-portal.org/smash/record.jsf?pid=diva2:1764099
4	Implementing Test Automation with Selenium WebDriver: Case Study MeetingPackage.com Penulis: Quan Dao (2018)	Lahti University of Applied Sciences (LAMK), Finlandia (Bachelor's Thesis in Business Information Technology)	Fokus: Pengukuran efisiensi waktu (time efficiency) penerapan otomatisasi pengujian menggunakan Selenium pada pengembangan perangkat lunak Agile/Scrum di perusahaan MeetingPackage.com. Hasil: Menggunakan metodologi Design Science Research untuk membangun artefak test suite Selenium. Evaluasi statistik terhadap konsumsi waktu Quality Assurance (QA) sebelum dan sesudah implementasi menunjukkan penurunan waktu pengujian regresi yang signifikan serta memberikan panduan struktural implementasi Selenium bagi tim QA. Tautan: https://www.theseus.fi/handle/10024/152509
5	A Search-Based Approach for Robustness Testing of Web Applications	Blekinge Institute of Technology (BTH), Swedia (Master's Thesis	Fokus: Pengujian ketahanan (robustness testing) aplikasi web lintas browser menggunakan Selenium yang dipadukan dengan algoritma

	Penulis: Master's Thesis Research Team — Blekinge Institute of Technology (2019)	in Software Engineering, bekerja sama dengan Ericsson)	evolusioner NSGA-II (Search-Based Software Testing). Hasil: Membangun testbed eksperimental di Ericsson yang memanfaatkan log akses pengguna dan mengotomatisasi browser via Selenium untuk membangkitkan urutan pengujian secara otomatis. Pendekatan ini berhasil mengungkap cacat perangkat lunak (fault revelation) dengan panjang urutan tes yang optimal dan menurunkan biaya pemeliharaan pengujian. Tautan: https://www.diva-portal.org/smash/get/diva2:1336628/FULLTEXT02
6	Comparative Analysis of Selecting a Test Automation Framework for an E-Commerce Website Penulis: K. H. Brahmbhatt (2023)	Tallinn University of Technology (TalTech), Estonia (Master's Thesis)	Fokus: Analisis pemilihan kerangka kerja otomatisasi pengujian terbaik untuk skenario transaksi pada situs web e-commerce berskala modern. Hasil: Mengevaluasi arsitektur Selenium terhadap framework alternatif pada skenario alur pencarian produk, keranjang belanja, dan checkout. Penelitian menyimpulkan matriks keputusan pemilihan tool berdasarkan skala tim, kemudahan penggunaan, serta stabilitas eksekusi lintas browser. Tautan: https://digikogu.taltech.ee/

BAB III: IMPLEMENTASI SAMPLE SEDERHANA MENGGUNAKAN SELENIUM IDE

Pada implementasi sample sederhana ini, pengujian otomatis dilakukan menggunakan aplikasi resmi Selenium IDE Client (metode Record & Playback). Pendekatan ini dipilih karena mampu memperlihatkan secara visual bagaimana interaksi pengguna pada halaman web direkam menjadi instruksi perintah Selenese (Command, Target, Value) dan dieksekusi kembali secara otomatis tanpa intervensi manual. Objek pengujian menggunakan situs web e-commerce latihan standar QA yaitu SauceDemo (https://www.saucedemo.com).

3.1 Skenario Pengujian (Test Case Scenario)

- Nama Test Case: TC_01_Login_dan_Tambah_Keranjang
- URL Target (Base URL): https://www.saucedemo.com
- Tujuan Pengujian: Memvalidasi bahwa pengguna dapat masuk (login) menggunakan akun valid dan berhasil menambahkan produk ke dalam keranjang belanja secara otomatis.
- Data Uji (Test Data): Username = standard_user | Password = secret_sauce

3.2 Langkah-Langkah Penggunaan & Perekaman pada Selenium IDE

1. Membuka Aplikasi & Membuat Proyek Baru: Buka aplikasi Selenium IDE Client. Pada jendela awal (Welcome to the Selenium IDE client), klik tombol 'CREATE PROJECT'.

2. Mengatur URL Target: Masukkan alamat target pengujian yaitu https://www.saucedemo.com pada kolom 'Project Base URL' di bagian atas aplikasi.

3. Memulai Perekaman (Start Recording): Klik tombol 'Record' (ikon lingkaran merah) pada toolbar kanan atas. Selenium IDE akan otomatis membuka jendela browser yang mengarah ke halaman login Swag Labs (SauceDemo).

4. Melakukan Simulasi Aksi Pengguna di Browser:
   - Klik pada kolom input Username, lalu ketikkan: standard_user
   - Klik pada kolom input Password, lalu ketikkan: secret_sauce
   - Klik tombol 'Login' sehingga halaman berpindah ke katalog produk (Products).
   - Klik tombol 'Add to cart' pada produk 'Sauce Labs Backpack'.
   - Klik ikon keranjang belanja di pojok kanan atas untuk melihat produk yang berhasil masuk.

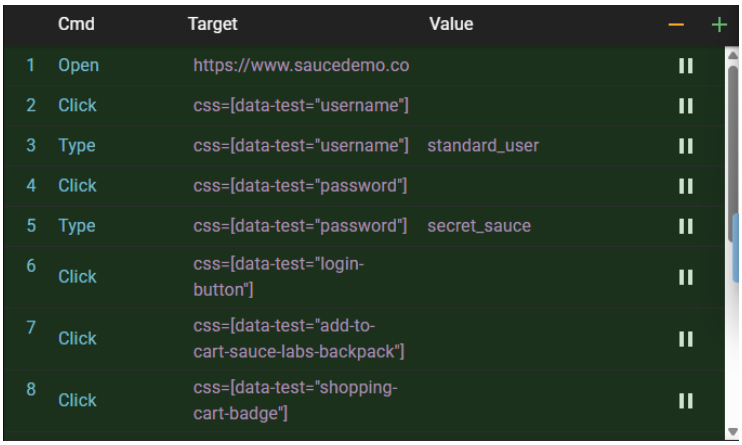
5. Menghentikan Perekaman (Stop Recording): Kembali ke jendela aplikasi Selenium IDE, lalu klik tombol 'Stop Recording' (ikon kotak merah). Seluruh aktivitas klik dan ketikan di browser otomatis tercatat menjadi tabel langkah perintah.

6. Menjalankan Pengujian Otomatis (Playback / Run Test): Klik tombol 'Play / Run Current Test' (ikon segitiga di bagian kiri atas). Tanpa menyentuh mouse atau keyboard, Selenium IDE akan membuka browser dan mengulangi seluruh langkah dari awal hingga akhir secara otomatis.

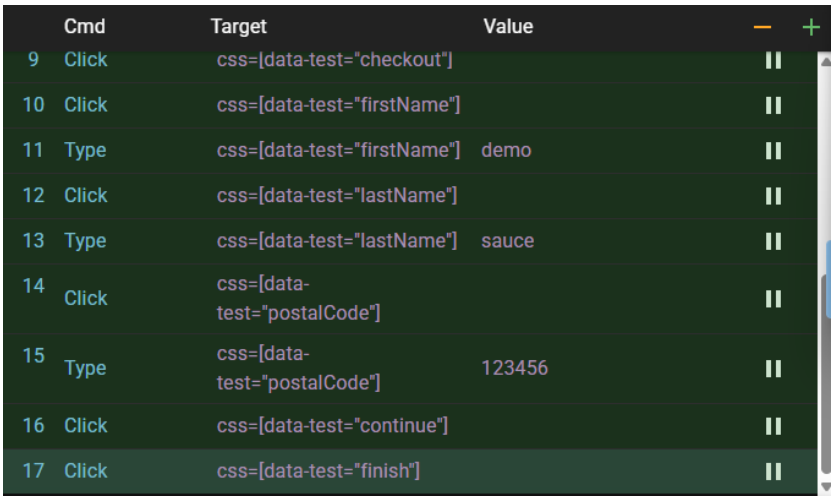
7. Menyimpan Proyek: Simpan proyek pengujian melalui menu File -> Save Project dengan nama file 'Tugas_Sample_Selenium.side'.

3.3 Tabel Instruksi Hasil Rekaman di Selenium IDE (Command, Target, Value)

Setelah proses perekaman selesai dilakukan, berikut adalah daftar instruksi otomatis yang terbentuk di dalam tabel editor tengah pada aplikasi Selenium IDE beserta status eksekusinya saat dijalankan (Playback):



	Cmd	Target	Value
1	Open	https://www.saucedemo.co	
2	Click	css=[data-test="username"]	
3	Type	css=[data-test="username"]	standard_user
4	Click	css=[data-test="password"]	
5	Type	css=[data-test="password"]	secret_sauce
6	Click	css=[data-test="login-button"]	
7	Click	css=[data-test="add-to-cart-sauce-labs-backpack"]	
8	Click	css=[data-test="shopping-cart-badge"]	



	Cmd	Target	Value
9	Click	css=[data-test="checkout"]	
10	Click	css=[data-test="firstName"]	
11	Type	css=[data-test="firstName"]	demo
12	Click	css=[data-test="lastName"]	
13	Type	css=[data-test="lastName"]	sauce
14	Click	css=[data-test="postalCode"]	
15	Type	css=[data-test="postalCode"]	123456
16	Click	css=[data-test="continue"]	
17	Click	css=[data-test="finish"]	